

Chapter 24: Multidisciplinary Optimization

Algorithms for Optimization, 2nd ed. (2026)
Kochenderfer & Wheeler

Lecture Notes

June 13, 2026

Chapter Outline

- 1 Introduction to Multidisciplinary Design Optimization
- 2 Disciplinary Analyses and Assignments
- 3 Interdisciplinary Compatibility and the MDA
- 4 MDO Architectures: Overview and the Running Example
- 5 Multidisciplinary Design Feasible (MDF)
- 6 Sequential Optimization
- 7 Individual Discipline Feasible (IDF)
- 8 Collaborative Optimization (CO)
- 9 Simultaneous Analysis and Design (SAND)
- 10 Worked Example: Spring-Pendulum System
- 11 Architecture Comparison and Selection Guide
- 12 Summary
- 13 Exercises

What is Multidisciplinary Design Optimization? I

Real engineering systems — aircraft, autonomous vehicles, power grids, financial portfolios — are not designed one component at a time. Every component interacts with every other component, and the best choice for one discipline (say, the lightest wing structure) may be terrible for another (e.g., aerodynamic performance demands a thicker profile). **Multidisciplinary Design Optimization (MDO)** is the branch of optimization that takes all of these interactions seriously and seeks a system-level optimum rather than a collection of local discipline optima.

What Makes MDO Different from Ordinary Optimization

In a single-discipline problem you minimize $f(\mathbf{x})$ subject to constraints on $\mathbf{x}$ ($\mathbf{x}$, the vector of design variables). In an MDO problem the objective depends on *outputs* of several interacting analyses, and each analysis may depend on outputs of the others. This circular dependency is the central challenge.

The most important consequence of these interdependencies is that optimizing each discipline in isolation — first optimize the structure, then optimize the aerodynamics, and so on — generally does *not* produce the system-level optimum. The disciplines must be considered simultaneously.

What is Multidisciplinary Design Optimization? II

Everyday Analogy

Imagine co-designing a backpack and a bicycle: the backpack should be as small as possible, but its size affects the rider's aerodynamic drag, which in turn affects what gear ratio is optimal for the bicycle, which feeds back into how heavy the bicycle can be, which determines how the backpack should be attached. Every dimension affects every other; no single dimension can be optimized independently.

What is Multidisciplinary Design Optimization? III

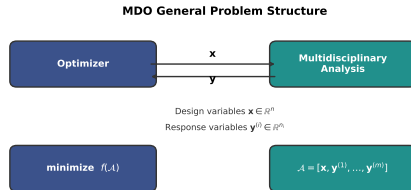
Core challenges in MDO:

Circular dependencies. Discipline A needs outputs from Discipline B, which needs outputs from Discipline A. This creates a system of equations that must be solved simultaneously, not sequentially.

Heterogeneous solvers. Different disciplines are often owned by different teams and use different software (finite-element codes, computational-fluid-dynamics solvers, market-simulation tools). The MDO architecture must orchestrate these without rewriting them.

Expensive function evaluations. A single call to a high-fidelity CFD or FEA solver can take hours. The MDO architecture must therefore minimise the number of such calls.

Large variable counts. Even simple systems have dozens of design variables per discipline; the combined problem



General MDO Problem Statement

$$\min_{\mathcal{A}} f(\mathcal{A}) \quad \text{subject to} \quad \mathcal{F}_i(\mathcal{A}) = \mathcal{A}, \forall i$$

where $\mathcal{A}$ (the *assignment*) collects all design and response variables, and the constraints say that running every disciplinary analysis $\mathcal{F}_i$ (script $\mathcal{F}$ sub i) must leave the

What is Multidisciplinary Design Optimization? IV

Key Insight

The feasibility condition $\mathcal{F}_i(\mathcal{A}) = \mathcal{A}$ is simply a mathematical way of saying “the disciplines agree with each other.” Finding such an assignment — let alone the best one — is what MDO architectures are designed to accomplish.

Disciplinary Analyses I

To describe MDO precisely, we need two foundational concepts: the *assignment* and the *disciplinary analysis*.

Definition: Assignment $\mathcal{A}$

An **assignment** $\mathcal{A}$ is a dictionary (a mapping from variable names to numerical values) that records the current values of *all* variables in the system: the design variables $\mathbf{x}$ (the quantities we are free to choose) and the response variables $\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(m)}$ (quantities computed by each of the m disciplines). The total size of the assignment is $|\mathcal{A}| = n + \sum_{i=1}^m n_i$, where n is the number of design variables and n_i (n sub i) is the number of response variables produced by discipline i .

Disciplinary Analyses II

Definition: Disciplinary Analysis $\mathcal{F}_i$

A **disciplinary analysis** $\mathcal{F}_i$ (script F sub i) is a function that reads the current assignment and updates (overwrites) the response variables of discipline i :

$$\mathbf{y}^{(i)} \leftarrow \mathcal{F}_i(\mathbf{x}, \mathbf{y}^{(-i)})$$

where $\mathbf{y}^{(-i)}$ (y superscript minus i) denotes all response variables *except* those of discipline i . In practice $\mathcal{F}_i$ is often a black-box simulation (a finite-element solver, a routing algorithm, a pricing model, ...).

The arrow $\leftarrow$ means “is assigned the value of”. After running $\mathcal{F}_i$, the variable $\mathbf{y}^{(i)}$ in the shared assignment dictionary holds the new, updated value.

Disciplinary Analyses III

Example 24.1: Three-Discipline System

Consider a system with one design variable x and three disciplines, each producing one response variable:

$$y^{(1)} \leftarrow \mathcal{F}_1(x, y^{(2)}, y^{(3)}) = y^{(2)} - x$$

$$y^{(2)} \leftarrow \mathcal{F}_2(x, y^{(1)}, y^{(3)}) = \sin(y^{(1)} + y^{(3)})$$

$$y^{(3)} \leftarrow \mathcal{F}_3(x, y^{(1)}, y^{(2)}) = \cos(x + y^{(1)} + y^{(2)})$$

Notice that $\mathcal{F}_1$ needs $y^{(2)}$, $\mathcal{F}_2$ needs $y^{(1)}$ and $y^{(3)}$, and $\mathcal{F}_3$ needs $y^{(1)}$ and $y^{(2)}$. These are *circular* dependencies: no discipline can be run “first” without some assumption about the others.

What this means concretely. Suppose we start with the initial guess $x = 1, y^{(1)} = y^{(2)} = y^{(3)} = 1$. Running $\mathcal{F}_1$ gives $y^{(1)} = 1 - 1 = 0$. Running $\mathcal{F}_2$ then gives $y^{(2)} = \sin(0 + 1) \approx 0.841$. Running $\mathcal{F}_3$ gives $y^{(3)} = \cos(1 + 0 + 0.841) \approx -0.297$. We are not done: the new $y^{(3)}$ would change $y^{(2)}$, which would change $y^{(1)}$, and so on. We must iterate until all three disciplines agree simultaneously.

Disciplinary Analyses IV

Key Insight

A disciplinary analysis is a function, not a solve. It takes the current state of the whole system (the assignment) and returns a *new* value for its own outputs. The challenge is to find an assignment where running any analysis leaves the assignment unchanged — the concept of *interdisciplinary compatibility*.

Python: Implementing Disciplinary Analyses (Example 24.1)

The simplest way to implement disciplinary analyses in Python is to represent the assignment as a dictionary and have each analysis function mutate the relevant entries in place.

```
import numpy as np

# Each disciplinary analysis takes the shared assignment dict,
# reads what it needs, and writes its own response variables.

def F1(A):
    """Discipline 1:  $y^{(1)} = y^{(2)} - x$ ."""
    A['y1'] = A['y2'] - A['x']
    return A

def F2(A):
    """Discipline 2:  $y^{(2)} = \sin(y^{(1)} + y^{(3)})$ ."""
    A['y2'] = np.sin(A['y1'] + A['y3'])
    return A

def F3(A):
    """Discipline 3:  $y^{(3)} = \cos(x + y^{(1)} + y^{(2)})$ ."""
    A['y3'] = np.cos(A['x'] + A['y2'] + A['y1'])
    return A

#         initial assignment for  $x = 1$ , all response vars = 1

A = {'x': 1.0, 'y1': 1.0, 'y2': 1.0, 'y3': 1.0}
```

Interdisciplinary Compatibility I

We now give a precise definition of what it means for an assignment to be *self-consistent*, i.e., for all disciplines to agree with one another.

Definition: Interdisciplinary Compatibility

An assignment $\mathcal{A}$ is called **interdisciplinarily compatible** (or simply *feasible*) if running every disciplinary analysis leaves the assignment unchanged:

$$\mathcal{F}_i(\mathcal{A}) = \mathcal{A} \quad \text{for all } i = 1, \dots, m$$

Equivalently, the *disciplinary residual* $R_i(\mathcal{A}) \equiv \mathcal{F}_i(\mathcal{A}) - \mathcal{A}$ equals zero for every discipline i . Here “R” stands for residual: the amount by which the discipline’s output *differs* from what is currently stored in the assignment.

A compatible assignment is the analogue of a fixed point of the system of equations. At such a point, if you ask any discipline “given the current values of everything else, what would you compute for your own outputs?” it answers with the values already stored in the assignment. There is no more updating to do.

Interdisciplinary Compatibility II

To find a compatible assignment we use a process called a **Multidisciplinary Analysis (MDA)**, which is simply an iterative algorithm that keeps running the disciplinary analyses in turn until they stop changing the assignment.

Interdisciplinary Compatibility III

Dependency Graphs

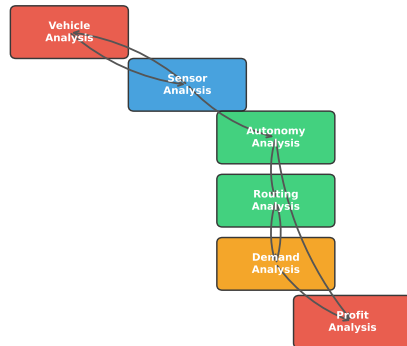
The interaction structure of a multidisciplinary system is best visualised as a **directed graph**, where:

- Each node represents a discipline (or a variable).
- A directed edge $i \rightarrow j$ means “discipline j requires the output of discipline i .”

If the graph has **no cycles**, the analyses can be run in topological order (a sequence where each discipline's inputs are already available) and the system is solved in a single pass. If the graph has **cycles**, iterative solution (an MDA) is unavoidable.

Topological sorting is a standard graph algorithm that, given a directed acyclic graph (DAG), produces a linear ordering of the nodes such that every edge

Ride-Sharing Problem: Dependency Graph



Dependency graph for the ride-sharing problem (introduced later). The two-way arrows between Vehicle and Sensor, and between Routing and Demand, indicate cycles that require iterative solution.

Key Insight

Before choosing an MDO architecture, always draw the dependency graph. If there are no cycles (rare in complex engineering systems) a single forward pass suffices. If there are cycles, the choice of architecture determines how those cycles are resolved — and that choice has major implications for computational cost.

The Gauss-Seidel MDA Algorithm I

The simplest and most widely used iterative MDA algorithm is the **Gauss-Seidel** (pronounced “gows-ZY-del”) method, named after the same iterative scheme used to solve systems of linear equations. The idea is straightforward: cycle through the disciplinary analyses repeatedly, each time updating the assignment with the latest values, until the assignment stops changing.

The Gauss-Seidel MDA Algorithm II

Algorithm 24.1: Gauss-Seidel MDA

Inputs:

- $[F_1, \dots, F_m]$ — the list of disciplinary analysis functions, in the desired evaluation order.
- $\mathcal{A}$ — an initial assignment (an initial guess for all response variables; the design variable $\mathbf{x}$ is fixed for this call).
- $k_{\max}$ — the maximum number of outer iterations (a safeguard against infinite loops if the system does not converge).
- ε (epsilon) — the relative convergence tolerance: we declare convergence when no variable changes by more than a fraction ε of its current magnitude.

Procedure:

- 1 Set $k \leftarrow 0$, converged $\leftarrow$ False.
- 2 While not converged and $k \leq k_{\max}$:
 - 1 $k \leftarrow k + 1$.
 - 2 Save a deep copy: $\mathcal{A}_{\text{old}} \leftarrow \mathcal{A}$.
 - 3 For each discipline F_i in order: run $F_i(\mathcal{A})$, updating $\mathcal{A}$ in place.
 - 4 Check: set converged = True if $|v(\mathcal{A}) - v(\mathcal{A}_{\text{old}})| \leq \varepsilon \cdot |v(\mathcal{A}_{\text{old}})|$ for every numeric variable v .
- 3 Return ($\mathcal{A}$, converged)

Python: Gauss-Seidel MDA Implementation

The Python implementation below closely follows Algorithm 24.1. It works with any list of disciplinary analysis functions as long as each function accepts and mutates the shared assignment dictionary.

```
import copy, numpy as np

def gauss_seidel_mda(Fs, A, k_max=100, eps=1e-6):
    """
    Run the Gauss-Seidel MDA until convergence or k_max iterations.

    Parameters
    -----
    Fs      : list of callables, each of the form F(A) -> A (mutates in place)
    A       : dict, the shared assignment {'x': ..., 'y1': ..., ...}
    k_max   : int, maximum number of outer iterations
    eps     : float (epsilon), relative convergence tolerance

    Returns
    -----
    A       : dict, the (hopefully converged) assignment
    converged : bool, True iff the tolerance was met
    """
    k, converged = 0, False
    while not converged and k <= k_max:
        k += 1
        A_old = copy.deepcopy(A)          # save a snapshot before this iteration
        for F in Fs:
            F(A)                          # each analysis updates A in place
```

MDO Architectures: Overview I

Having defined what it means for an assignment to be feasible (interdisciplinarily compatible) and how to find such an assignment via the MDA, we now turn to the central question of this chapter: *how should the MDA be embedded inside an optimizer?*

Different answers to this question give rise to different **MDO architectures**. Each architecture is a blueprint that specifies:

- ① Which variables are optimiser decision variables (the things the optimiser directly controls).
- ② How and when the disciplinary analyses are called.
- ③ How coupling between disciplines is enforced (either through an inner MDA loop, through equality constraints, or through a bi-level structure).

The General MDO Problem

All architectures solve, in one form or another, the same underlying problem:

$$\min_{\mathcal{A}} f(\mathcal{A}) \quad \text{subject to} \quad \mathcal{F}_i(\mathcal{A}) = \mathcal{A}, \quad i = 1, \dots, m, \quad \mathcal{A} \in \mathcal{X}$$

where f (f) is the system-level objective (e.g., negative profit, total weight, total cost), $\mathcal{F}_i$ are the disciplinary analyses, and $\mathcal{X}$ encodes bounds and inequality constraints on the design and response variables.

MDO Architectures: Overview III

MDO Architecture Comparison

Architecture	Optimizer Level	MDA Required	Parallel Disciplines	Variable Count	Notes
MDF	Single	Yes (inner loop)	No	Low (n)	Simple; MDA cost dominates
Sequential	Hierarchical	Per discipline	No	Low-Med	Good for loosely coupled
IDF	Single	No	Yes	High ($n + \sum n_i$)	Coupling as constraint
CO	Bi-level	No	Yes	High	Each discipline independent
SAND	Single (large)	No (residuals)	Yes	Very high	Needs analytic gradients

This chapter covers five main architectures, each making a different trade-off between these factors:

Single-level architectures use one optimizer that directly controls the outcome:

- **Multidisciplinary Design Feasible (MDF):** the simplest approach; every function evaluation runs a full MDA.

- **Individual Discipline Feasible (IDF):** no

Hierarchical / distributed architectures split the problem across multiple levels or teams:

- **Sequential Optimization:** disciplines optimise one after another in a round-robin fashion.

- **Collaborative Optimization (CO):** each

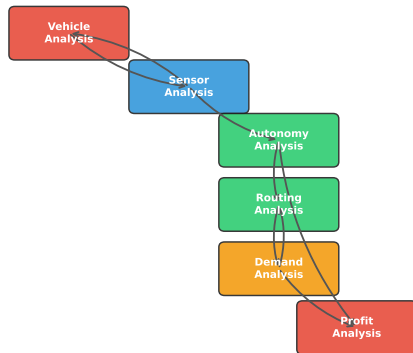
Running Example: The Ride-Sharing Company (Example 24.3) I

To make the comparison between architectures concrete, the textbook uses a single running example throughout the chapter: a hypothetical autonomous ride-sharing company that is *simultaneously* designing several interacting components of its business.

The four design vectors:

- **v** (vehicle): structural geometry, engine and drive train, battery capacity, passenger capacity, weight, drag.
- **s** (sensor package): the autonomous driving sensors (cameras, lidar, radar); must meet safety requirements that depend on the vehicle geometry.
- **r** (routing strategy): the algorithm that dispatches vehicles to passenger requests; performance depends on vehicle range and the number of simultaneous passengers.
- **p** (pricing scheme): the fare structure; affects passenger demand, which in turn affects routing performance.

Ride-Sharing Problem: Dependency Graph



Running Example: The Ride-Sharing Company (Example 24.3) II

The six disciplinary analyses:

- 1 **Vehicle Analysis:** given $\mathbf{v}$ and sensor weight from the sensor analysis, computes vehicle range, fuel efficiency, and weight.
- 2 **Sensor Analysis:** given vehicle geometry and performance from the vehicle analysis, computes sensor specifications and sensor weight.
- 3 **Autonomy Analysis:** given vehicle and sensor outputs, evaluates the autonomy level (how capable the self-driving system is under various conditions).
- 4 **Routing Analysis:** given autonomy level and passenger demand from the demand analysis, computes routing performance (average wait time, throughput).
- 5 **Demand Analysis:** given routing performance and the pricing scheme, estimates passenger demand.
- 6 **Profit Analysis:** given all other outputs plus vehicle and pricing design variables, computes net profit.

Why This Example is Challenging

The two cycles (Vehicle $\leftrightarrow$ Sensor and Routing $\leftrightarrow$ Demand) mean a full MDA is needed at every point evaluated. Each architecture handles these cycles differently, with different implications for how many times each analysis is called and what is left for the optimizer to control.

MDF: Multidisciplinary Design Feasible Architecture I

The **Multidisciplinary Design Feasible (MDF)** architecture is conceptually the simplest of all MDO architectures. Its central idea is to hide all the complexity of the coupled system *inside* a black-box function that the optimizer sees as an ordinary, single-discipline objective function.

The key idea. At every point $\mathbf{x}$ that the optimizer proposes, MDF runs a complete inner MDA loop (e.g., Gauss-Seidel) to find the unique compatible assignment $\mathcal{A}^*(\mathbf{x})$ such that all disciplines agree. Only once this inner convergence is achieved is the objective $f(\mathcal{A}^*(\mathbf{x}))$ returned to the optimizer.

MDF Problem Formulation

From the optimizer's perspective, MDF solves:

$$\min_{\mathbf{x} \in \mathcal{X}} f(\text{MDA}(\mathbf{x}))$$

where $\text{MDA}(\mathbf{x})$ (MDA stands for Multidisciplinary Analysis) is a function that, given design variables $\mathbf{x}$, runs the iterative Gauss-Seidel loop until convergence and returns the fully compatible assignment $\mathcal{A}^*$. The optimizer only sees a function of $\mathbf{x}$ and has no knowledge of the inner iteration; it treats MDA as a black-box simulation.

MDF: Multidisciplinary Design Feasible Architecture II

Because the MDA guarantees that the assignment is always compatible before the objective is evaluated, *every* iterate generated by the MDF optimizer is physically feasible. This is a major practical advantage: the design engineer can inspect any intermediate iterate and it corresponds to a real, self-consistent design.

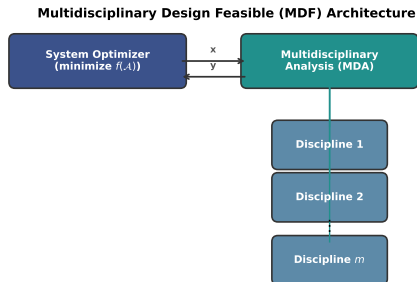
MDF: Multidisciplinary Design Feasible Architecture III

MDF procedure step by step:

- 1 The outer optimizer (e.g., gradient-based L-BFGS, or a derivative-free method) proposes a new design vector $\mathbf{x}^{(k)}$ ($\mathbf{x}$ superscript k , the value of $\mathbf{x}$ at iteration k).
- 2 An inner MDA loop (Gauss-Seidel or Newton's method) iterates until $\|\mathcal{A}_{\text{new}} - \mathcal{A}_{\text{old}}\| \leq \epsilon$, producing the compatible assignment $\mathcal{A}^*(\mathbf{x}^{(k)})$.
- 3 The objective $f(\mathcal{A}^*)$ is returned to the optimizer.
- 4 If gradient-based: compute $\nabla_{\mathbf{x}} f$ using the adjoint method or finite differences through the MDA.
- 5 The optimizer updates $\mathbf{x}^{(k+1)}$ and repeats.

Properties of MDF

- ✓ Small optimizer variable space: only $\mathbf{x}$ (not the response variables).
- ✓ Every iterate is interdisciplinarily compatible



The MDF architecture. The optimizer calls the MDA as a black box; the MDA calls the disciplinary analyses in a loop.

MDF: Multidisciplinary Design Feasible Architecture IV

Computational cost analysis. Let K (K) be the average number of Gauss-Seidel iterations required for MDA convergence, N_f (N sub f) be the number of function evaluations the outer optimizer requires, and C_i (C sub i) be the cost of one call to disciplinary analysis i . The total cost of MDF is approximately:

$$\text{Cost}_{\text{MDF}} \approx N_f \cdot K \cdot \sum_{i=1}^m C_i$$

If the outer optimizer needs $N_f = 200$ function evaluations, the MDA requires $K = 30$ iterations, and there are $m = 6$ analyses each costing 1 unit of time, the total cost is $200 \times 30 \times 6 = 36,000$ analysis calls. If each analysis call takes one hour (realistic for high-fidelity CFD), the total wall-clock time is 36,000 hours — clearly impractical. This is why alternative architectures are sought for expensive analyses.

MDF Applied to Ride-Sharing (Example 24.4)

For the ride-sharing problem, the MDF formulation is:

$$\min_{\mathbf{v}, \mathbf{s}, \mathbf{r}, \mathbf{p}} -\text{profit}(\text{MDA}(\mathbf{v}, \mathbf{s}, \mathbf{r}, \mathbf{p}))$$

subject to design constraints on $[\mathbf{v}, \mathbf{s}, \mathbf{r}, \mathbf{p}]$. The optimizer controls only the four design vectors; the six analyses run inside the MDA at each evaluation. This is conceptually very clean, and works well when the analyses are fast.

Python: MDF Implementation

```
import copy, numpy as np
from scipy.optimize import minimize

#         Reuse the disciplinary analyses from before

# F1, F2, F3 are defined in the previous code frame.

def run_mda(x_val, Fs, y_init=None, k_max=200, eps=1e-8):
    """
    Run the full Gauss-Seidel MDA for a fixed design variable x_val.

    Returns the converged assignment or raises RuntimeError if it fails.
    """
    A = {'x': float(x_val)}
    A.update(y_init or {'y1': 0.0, 'y2': 0.0, 'y3': 0.0})
    A, converged = gauss_seidel_mda(Fs, A, k_max=k_max, eps=eps)
    if not converged:
        raise RuntimeError(f'MDA did not converge for x={x_val}')
    return A

def mdf_objective(x_arr):
    """
    MDF objective: run the full MDA, then evaluate the system objective.
    The optimizer only sees this function; it has no knowledge of the MDA.
    """
    A = run_mda(x_arr[0], [F1, F2, F3])
    # Objective: minimize the sum of squared response variables
```

Sequential Optimization Architecture I

The **sequential optimization** architecture takes a different approach to decomposition: rather than having a single optimizer control all design variables at once, it partitions the design variables by discipline and lets each discipline's team run its own local optimizer in turn, in a round-robin fashion.

The key idea. Each discipline i has its own *local* design variables $\mathbf{x}_\ell^{(i)}$ (x subscript ℓ superscript i) — parameters that are only used within that discipline and do not directly appear in the objectives of other disciplines. There are also *global* design variables $\mathbf{x}_g$ (x subscript g) that are shared across disciplines and therefore controlled by a top-level outer optimizer.

Sequential Optimization Architecture II

Sequential Optimization: Problem Structure

Top-level optimizer controls $\mathbf{x}_g$:

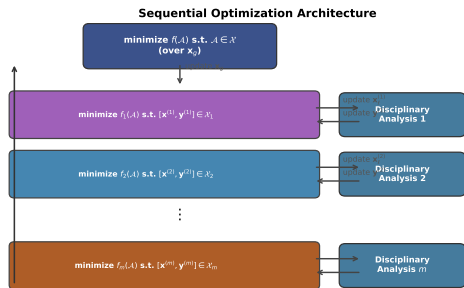
$$\min_{\mathbf{x}_g \in \mathcal{X}_g} f(\mathcal{A}) \quad \text{subject to} \quad \mathcal{A} \in \mathcal{X}$$

Subproblem i (run after fixing $\mathbf{x}_g$):

$$\min_{\mathbf{x}_\ell^{(i)}} f_i(\mathcal{A}) \quad \text{subject to} \quad [\mathbf{x}^{(i)}, \mathbf{y}^{(i)}] \in \mathcal{X}_i, \quad \mathcal{F}_i(\mathbf{x}_g, \mathbf{x}_\ell^{(i)}, \mathcal{A}) \text{ run}$$

After solving subproblem i , the local variables $\mathbf{x}_\ell^{(i)}$ and the response $\mathbf{y}^{(i)}$ are updated in $\mathcal{A}$ before discipline $i + 1$ runs its subproblem.

Sequential Optimization Architecture III



General sequential architecture with m discipline subproblems and a top-level global optimizer.

How the sequential architecture works:

- 1 The top-level optimizer proposes a new value for the global variables $\mathbf{x}_g$.
- 2 Subproblem 1 optimises its local variables $\mathbf{x}_\ell^{(1)}$ to minimise $f_1(\mathcal{A})$ subject to discipline-1 constraints; Disciplinary Analysis 1 is called to update $\mathbf{y}^{(1)}$.
- 3 Subproblem 2 receives the updated $\mathcal{A}$ and optimises $\mathbf{x}_\ell^{(2)}$ to minimise $f_2(\mathcal{A})$, calling Disciplinary Analysis 2 to update $\mathbf{y}^{(2)}$.
- 4 This continues through all m subproblems.
- 5 The system-level objective $f(\mathcal{A})$ is returned to the top-level optimizer.
- 6 Steps 1–5 repeat until top-level convergence.

Sequential Optimization Architecture IV

Example 24.5: Sequential Optimization for Ride-Sharing

Applied to the ride-sharing problem (Figure 24.8 in the book), the sequential architecture proceeds as follows in each outer iteration:

- ① Top-level optimizer updates global variables $\mathbf{v}_g, \mathbf{s}_g, \mathbf{r}_g, \mathbf{p}_g$ (vehicle capacity, sensor specifications, routing targets, pricing targets that feed into multiple disciplines).
- ② Vehicle subproblem: optimises local drive-train and geometry variables $\mathbf{v}_\ell$ while running Vehicle Analysis.
- ③ Sensor subproblem: optimises local sensor parameters $\mathbf{s}_\ell$ while running Sensor Analysis.
- ④ Autonomy subproblem: similarly for autonomy variables.
- ⑤ Routing subproblem: optimises local routing parameters.
- ⑥ Demand subproblem: optimises local demand/pricing parameters.
- ⑦ Profit Analysis runs (no local variables needed here) to compute the objective.

Note that the Routing–Demand cycle is addressed by their sequential ordering: Routing runs before Demand in each outer iteration, so changes in routing performance feed into demand during the *same* outer iteration, but the reverse effect (demand $\rightarrow$ routing) must wait for the next outer iteration.

IDF: Individual Discipline Feasible Architecture I

The **Individual Discipline Feasible (IDF)** architecture resolves the expensive inner MDA loop of MDF by taking a completely different approach to compatibility: instead of running the MDA to convergence at each function evaluation, it introduces *coupling variables* into the optimizer's decision space and uses *equality constraints* to enforce compatibility only at the solution.

The key idea. For each discipline i , create a vector of *coupling copies* $\mathbf{c}^{(i)}$ ($\mathbf{c}$ superscript i) that the optimizer controls directly. These coupling copies serve as *assumed values* for the response variables $\mathbf{y}^{(i)}$ when they appear as inputs to other disciplines. The equality constraints

$$\mathbf{c}^{(i)} = \mathcal{F}_i(\mathbf{x}, \mathbf{c}) \quad \forall i$$

(read: “coupling copy i must equal what Discipline i actually computes”) enforce interdisciplinary compatibility — but only at the *optimal point*, not at intermediate iterates.

This means each disciplinary analysis is called only *once per outer iteration*, with a copy of the full assignment. All disciplines can be called in parallel because they each receive a copy of $\mathcal{A}$ at the beginning of the iteration and do not depend on each other's updated outputs within that iteration.

IDF: Individual Discipline Feasible Architecture II

IDF Formulation

The optimizer's decision variables are the design variables *and* the coupling copies: $[\mathbf{x}, \mathbf{c}^{(1)}, \dots, \mathbf{c}^{(m)}]$.

$$\min_{\mathbf{x}, \mathbf{c}^{(1)}, \dots, \mathbf{c}^{(m)}} f(\mathbf{x}, \mathbf{c}^{(1)}, \dots, \mathbf{c}^{(m)})$$

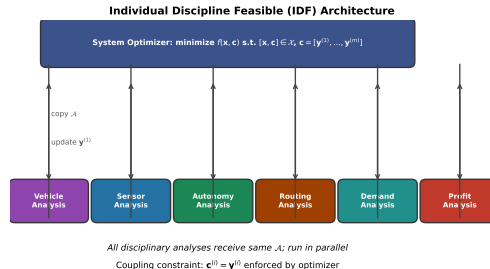
$$\text{subject to } [\mathbf{x}, \mathbf{c}] \in \mathcal{X}$$

$$\mathbf{c}^{(i)} = \mathcal{F}_i(\mathbf{x}, \mathbf{c}) \quad \forall i = 1, \dots, m$$

The last set of equations are $\sum_i n_i$ *equality constraints* that the optimizer must satisfy. At convergence, $\mathbf{c}^{(i)} = \mathbf{y}^{(i)}$ for all i , so the assignment is compatible.

Total number of optimizer variables:

$$|\text{IDF variables}| = n + \sum_{i=1}^m n_i$$



IDF architecture applied to the ride-sharing problem. Each analysis receives a copy of the full assignment and runs in parallel. The optimizer enforces coupling via equality constraints.

IDF: Individual Discipline Feasible Architecture III

Properties of IDF

- ✓ **Parallel execution:** since all disciplines receive a copy of $\mathcal{A}$ and run independently, they can be evaluated simultaneously on different compute nodes.
- ✓ **No inner MDA loop:** each outer optimizer iteration costs exactly m analysis calls (one per discipline), regardless of how tightly coupled the system is.
- ✓ **Works with gradient-based optimizers:** gradients of the coupling constraints $\partial \mathbf{c}^{(i)} / \partial \mathbf{x}$ can be computed via forward or adjoint methods.
- × **Large variable space:** the optimizer must handle $n + \sum n_i$ variables plus $\sum n_i$ equality constraints. For systems with many coupling variables this can be very large.
- × **Infeasible intermediates:** intermediate iterates are *not* interdisciplinarily compatible; the design is only feasible at the solution.

IDF: Individual Discipline Feasible Architecture IV

IDF for Ride-Sharing (Example 24.6)

In the ride-sharing IDF formulation, the optimizer controls $[\mathbf{v}, \mathbf{s}, \mathbf{r}, \mathbf{p}, \mathbf{c}^{(v)}, \mathbf{c}^{(s)}, \mathbf{c}^{(a)}, \mathbf{c}^{(r)}, \mathbf{c}^{(d)}, \mathbf{c}^{(p)}]$ (design vectors plus one coupling copy per analysis). Six equality constraints enforce $\mathbf{c}^{(i)} = \mathcal{F}_i(\mathbf{v}, \mathbf{s}, \mathbf{r}, \mathbf{p}, \mathbf{c})$ for each analysis i . All six analyses can run in parallel since each gets a copy of the same $\mathcal{A}$ and does not need to wait for the others.

Python: IDF Formulation and Implementation

```
import numpy as np
from scipy.optimize import minimize, NonlinearConstraint

# IDF decision vector:  $z = [x, y1_c, y2_c, y3_c]$ 
# where  $y1_c, y2_c, y3_c$  are the coupling copies ( $c^{(1)}, c^{(2)}, c^{(3)}$ ).

def idf_objective(z):
    """System objective as a function of design + coupling variables."""
    x, y1c, y2c, y3c = z
    return y1c**2 + y2c**2 + y3c**2    # minimise sum of squared responses

def idf_coupling_residuals(z):
    """
    Coupling residuals:  $c^{(i)} - F_i(x, c) = 0$  for each  $i$ .

    Discipline  $i$  uses the coupling copies as inputs, computes its output,
    and the constraint enforces that the output matches the coupling copy.
    """
    x, y1c, y2c, y3c = z
    # Discipline 1:  $y^{(1)} = y^{(2)} - x$  --> residual:  $y1c - (y2c - x)$ 
    r1 = y1c - (y2c - x)
    # Discipline 2:  $y^{(2)} = \sin(y^{(1)} + y^{(3)})$  --> residual:  $y2c - \sin(y1c+y3c)$ 
    r2 = y2c - np.sin(y1c + y3c)
    # Discipline 3:  $y^{(3)} = \cos(x+y^{(2)}+y^{(1)})$  --> residual:  $y3c - \cos(x+y2c+y1c)$ 
    r3 = y3c - np.cos(x + y2c + y1c)
    return np.array([r1, r2, r3])
```

Collaborative Optimization (CO) I

The **Collaborative Optimization (CO)** architecture is designed for situations where different disciplines are developed and maintained by *separate, autonomous teams*, each with its own preferred optimization tools, design space, and local constraints. CO is a *bi-level* architecture: there is a system-level optimizer that sets targets for shared (global) variables, and each discipline has its own *sub-optimizer* that tries to match those targets as closely as possible.

The key idea. The system-level optimizer specifies a “global assignment” $\mathcal{A}_g$ (A subscript g) — target values for all shared design and response variables. Each discipline i then runs its own local optimizer to find the combination of its own variables that (a) satisfies all of its local constraints and (b) minimises a *coupling objective* $J^{(i)}$ (J superscript i) that measures how far the discipline’s outputs deviate from the system-level targets. Interdisciplinary compatibility is achieved when every discipline can match the targets exactly, i.e., $J^{(i)} = 0$ for all i .

Collaborative Optimization (CO) II

CO System-Level Problem

$$\min_{\mathcal{A}_g} f(\mathcal{A}_g) \quad \text{subject to} \quad J^{(i)}(\mathcal{A}_g) \leq \delta, \quad i = 1, \dots, m$$

where δ (delta) is a small positive tolerance (ideally zero, but set to a small positive number for numerical stability), and the constraint $J^{(i)} \leq \delta$ says “discipline i must be able to match the system targets to within tolerance δ .”

Collaborative Optimization (CO) III

CO Sub-Problem for Discipline i

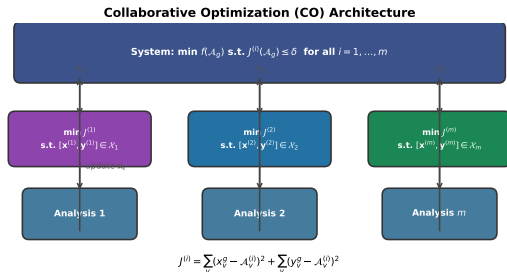
Each discipline i solves:

$$J_{\star}^{(i)}(\mathcal{A}_g) = \min_{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}} \underbrace{\sum_{v \in \mathbf{x}_g} (x_v^g - \mathcal{A}_v^{(i)})^2 + \sum_{v \in \mathbf{y}_g} (y_v^g - \mathcal{A}_v^{(i)})^2}_{J^{(i)}: \text{sum of squared deviations from targets}} \quad \text{s.t.} \quad [\mathbf{x}^{(i)}, \mathbf{y}^{(i)}] \in \mathcal{X}_i$$

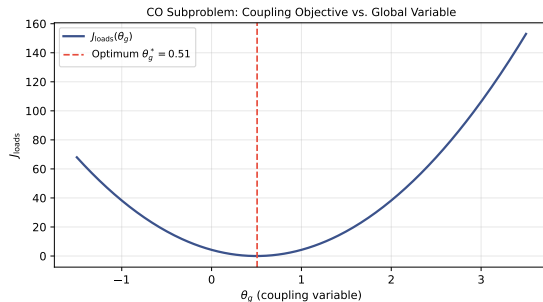
Reading this formula: x_v^g (x sub v superscript g) is the system-level target for shared design variable v ; $\mathcal{A}_v^{(i)}$ is discipline i 's local value for that same variable. The first sum penalises discrepancy in shared design variables; the second sum penalises discrepancy in shared response variables. The discipline team minimises this discrepancy using their own local solver and subject to their own local constraints $\mathcal{X}_i$.

A critical observation: the sub-problems for different disciplines are *completely independent of one another* — they share no variables and impose no constraints on each other. This means all sub-problems can be solved *in parallel*, which is a major advantage for distributed teams.

Collaborative Optimization (CO) IV



Coupling: each subproblem drives local values to match system-level targets



The coupling objective $J^{(i)}$ as a function of a global variable θ_g (theta sub g). When the system target θ_g is achievable by the discipline, $J^{(i)} = 0$. When it is outside the discipline's feasible region, $J^{(i)} > 0$.

CO architecture. The system optimizer sets global targets A_g ; each discipline sub-optimizer minimises its coupling objective $J^{(i)}$ independently.

Properties of CO

- ✓ Maximum **disciplinary autonomy**: each team uses its own tools, optimizers, and constraints without exposing implementation details to other teams.
- ✓ Sub-problems run **in parallel**.
- ✗ The bi-level structure creates a numerically challenging outer problem: $J_{\star}^{(i)}(\mathcal{A}_g)$ may be non-smooth and expensive to evaluate (it involves an inner optimization).
- ✗ Convergence of the outer loop is not guaranteed.

Example 24.7: CO Applied to Ride-Sharing

The system-level optimizer controls the global shared targets: vehicle capacity, sensor weight limit, routing throughput target, and demand target, among others.

- **Vehicle sub-problem:** finds $\mathbf{v}_\ell$ that minimises discrepancy between its computed outputs and the system targets (e.g., vehicle weight and range as close to $\mathcal{A}_g$ as possible), subject to structural and battery constraints.
- **Sensor sub-problem:** finds $\mathbf{s}_\ell$ that minimises discrepancy, subject to safety-requirement constraints.
- **Demand sub-problem:** finds pricing parameters that minimise discrepancy between predicted demand and the target demand, subject to market feasibility constraints.

The system optimizer then adjusts $\mathcal{A}_g$ to minimise negative profit subject to all $J^{(i)} \leq \delta$.

Python: Collaborative Optimization Sub-Problems

```
import numpy as np
from scipy.optimize import minimize

#         Example: CO sub-problems for the three-discipline system

# The system-level targets are A_g = {'x': x_g, 'y1_g': ..., 'y2_g': ..., 'y3_g': ...}

def co_subproblem_1(A_g):
    """
    Discipline 1 sub-problem: find y1 that minimises J^(1).
    J^(1) = (y1 - y1_g)^2 subject to: y1 = y2_g - x_g (discipline equation)
    Since y1 is fully determined by (y2_g - x_g), J^(1) measures how well
    the discipline equation can be satisfied.
    """
    y1_computed = A_g['y2_g'] - A_g['x_g']
    J1 = (y1_computed - A_g['y1_g'])**2
    return J1, y1_computed

def co_subproblem_2(A_g):
    """
    Discipline 2 sub-problem: finds y2 that minimises J^(2).
    y2 = sin(y1_g + y3_g); J^(2) = (y2 - y2_g)^2
    """
    y2_computed = np.sin(A_g['y1_g'] + A_g['y3_g'])
    J2 = (y2_computed - A_g['y2_g'])**2
    return J2, y2_computed
```

SAND: Simultaneous Analysis and Design I

The **Simultaneous Analysis and Design (SAND)** architecture (also sometimes called *All-at-Once*, or AAO) is the most mathematically direct formulation of the MDO problem. It treats both the design variables $\mathbf{x}$ and the response variables $\mathbf{y}$ as free optimization variables, and adds the disciplinary equations as explicit equality constraints.

The key idea. In both MDF and IDF, there is some notion of “running” a disciplinary analysis as a function. In SAND, we do not “run” the analyses at all during optimization. Instead, we write each analysis as a *residual equation*:

$$R_i(\mathbf{x}, \mathbf{y}) \equiv \mathcal{F}_i(\mathbf{x}, \mathbf{y}) - \mathbf{y}^{(i)} = \mathbf{0}$$

(R sub i , the residual of discipline i , equals the analysis output minus the response variable stored in the assignment.) These residual equations become equality constraints in the optimization problem, and the optimizer drives them to zero *simultaneously* with minimising the objective. There is no inner loop at all; the analysis and design happen at the same time.

SAND: Simultaneous Analysis and Design II

SAND Problem Formulation

$$\min_{\mathbf{x}, \mathbf{y}^{(1)}, \dots, \mathbf{y}^{(m)}} f(\mathbf{x}, \mathbf{y})$$

$$\text{subject to } R_i(\mathbf{x}, \mathbf{y}) = \mathbf{0}, \quad i = 1, \dots, m, \quad [\mathbf{x}, \mathbf{y}] \in \mathcal{X}$$

where $\mathbf{y} = [\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(m)}]$ collects all response variables, and there are $\sum_{i=1}^m n_i$ equality constraints in total.

SAND: Simultaneous Analysis and Design III

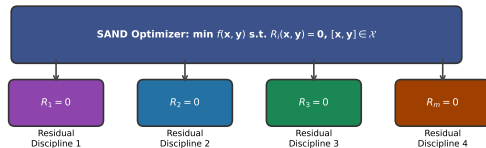
How SAND works in practice. A gradient-based optimizer (typically an interior-point or SQP method) simultaneously updates $\mathbf{x}$ and $\mathbf{y}$ at each iteration. At each iterate, the residuals R_i are generally nonzero — the system is *infeasible* during optimization. Only at convergence do all residuals reach zero and the assignment become compatible.

SAND is most efficient when analytic (or adjoint-based) Jacobians $\partial R_i / \partial \mathbf{x}$ and $\partial R_i / \partial \mathbf{y}$ are available. With these, the optimizer can take large Newton-like steps rather than small finite-difference steps. This is the basis of high-performance aerodynamic shape optimization codes.

Properties of SAND

- ✓ No inner MDA loop; each iteration calls each analysis exactly once.
- ✓ Fastest per iteration cost (one analysis per

Simultaneous Analysis and Design (SAND) Architecture



All design $\mathbf{x}$ and response $\mathbf{y}$ variables passed simultaneously to optimizer

Residuals $R_i(\mathbf{x}, \mathbf{y}) = 0$ enforced as equality constraints

Enables gradient-based methods; disciplinary analyses run once per iteration

SAND architecture for the ride-sharing problem. All design and response variables are optimized simultaneously; disciplinary residuals are explicit equality constraints.

SAND vs. MDF: Key Difference

In MDF, the assignment is always compatible

SAND: Simultaneous Analysis and Design IV

SAND Applied to Ride-Sharing

In the SAND formulation for the ride-sharing problem, the optimizer controls $[\mathbf{v}, \mathbf{s}, \mathbf{r}, \mathbf{p}, \mathbf{y}^{(v)}, \mathbf{y}^{(s)}, \mathbf{y}^{(a)}, \mathbf{y}^{(r)}, \mathbf{y}^{(d)}, \mathbf{y}^{(p)}]$ simultaneously. Six residual constraints are added:

$$R_v(\mathbf{v}, \mathbf{s}, \mathbf{y}) = \mathbf{0} \quad (\text{Vehicle Analysis residual})$$

$$R_s(\mathbf{s}, \mathbf{v}, \mathbf{y}) = \mathbf{0} \quad (\text{Sensor Analysis residual})$$

$$\vdots$$

$$R_p(\mathbf{p}, \mathbf{y}) = \mathbf{0} \quad (\text{Profit Analysis residual})$$

The interior-point or SQP optimizer drives all six residuals to zero simultaneously with maximizing profit. An adjoint code that computes $\partial R_i / \partial \mathbf{x}$ and $\partial R_i / \partial \mathbf{y}$ analytically makes this highly efficient.

Python: SAND Formulation and Implementation

```
import numpy as np
from scipy.optimize import minimize, NonlinearConstraint

# SAND decision vector: z = [x, y1, y2, y3]
# All four values are free; residuals are equality constraints.

def sand_objective(z):
    """System objective: minimise sum of squared response variables."""
    x, y1, y2, y3 = z
    return y1**2 + y2**2 + y3**2

def sand_residuals(z):
    """
    Disciplinary residuals  $R_i = F_i(x, y) - y_i$  for each discipline.

    At the solution these must all be zero; during iteration they are nonzero.
    """
    x, y1, y2, y3 = z
    R1 = (y2 - x) - y1          # R_1: F_1 output minus y1
    R2 = np.sin(y1 + y3) - y2   # R_2: F_2 output minus y2
    R3 = np.cos(x + y2 + y1) - y3 # R_3: F_3 output minus y3
    return np.array([R1, R2, R3])

def sand_jacobian(z):
    """
    Analytic Jacobian of the residuals with respect to  $z = [x, y1, y2, y3]$ .
    """
```

Spring-Pendulum Worked Example I

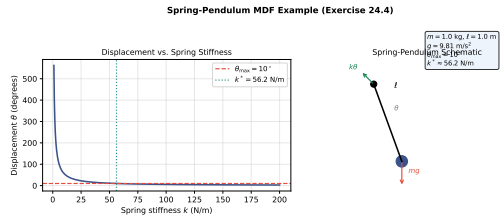
The textbook uses a **spring-pendulum** system as a concrete, numerically tractable example to demonstrate all four MDO architectures (MDF, IDF, CO, and SAND). This example is modelled after the problem of designing an aircraft wing: the loads (forces and moments) depend on the wing's deformation, and the deformation depends on the loads — a classic structural coupling.

Physical setup. A rigid pendulum of length ℓ (ell, the pendulum arm length, in metres) and point mass m (mass in kg) is held by a torsional spring of stiffness k (k , in N/m). Gravity g (gravitational acceleration, 9.81 m/s^2) deflects the pendulum by angle θ (theta, in radians) from the vertical.

Design goal. Minimise the spring stiffness k (cheaper, lighter spring) such that the deflection angle θ does not exceed $\theta_{\max}$ (theta subscript max, the allowed deflection limit), here set to 10° .

The two disciplinary analyses:

- 1 **Loads Analysis:** computes the gravitational moment M (in N·m) acting on the pendulum:



Circular Dependency

Notice the coupling: the Loads Analysis needs θ (to evaluate $\cos \theta$), and the Displacement Analysis produces θ . This is the same circular structure as in aircraft wing

Spring-Pendulum Worked Example II

Finding the compatible assignment (MDA). Given k , the MDA must find θ and M such that both equations hold simultaneously:

$$M = mg\ell \cos(\theta) \quad \text{and} \quad \theta = M/k$$

Substituting the second into the first gives $\theta = mg\ell \cos(\theta)/k$, i.e., $k\theta = mg\ell \cos(\theta)$. For $m = 1$, $\ell = 1$, $g = 9.81$, $k = 55.353$: $55.353\theta = 9.81 \cos(\theta)$. The Gauss-Seidel iterations starting at $\theta_0 = 0.1$ converge in about 20 iterations.

Spring-Pendulum Worked Example III

Worked Numerical Trace: MDA for $k = 55.353 \text{ N/m}$

Iteration	θ (rad)	M (N·m)	Change in θ
0 (init)	0.10000	—	—
1	$M_0/k = 9.81 \cos(0.1)/55.353 = 0.17647$	9.754	0.07647
2	$M_1/k = 9.81 \cos(0.17647)/55.353 = 0.17549$	9.735	0.00098
3	0.17550	9.736	$< 10^{-5}$

Converged to $\theta^* \approx 0.17550 \text{ rad} = 10.056^\circ \approx 10^\circ$ (at the constraint boundary), with $M^* \approx 9.736 \text{ N} \cdot \text{m}$ and $k^* \approx 55.353 \text{ N/m}$.

Spring-Pendulum Worked Example IV

MDF Formulation for Spring-Pendulum (Exercise 24.4)

From the optimizer's perspective, MDF controls only k (one design variable) and wraps the MDA in the objective:

$$\min_{k>0} f(\text{MDA}(k)) \quad \text{subject to} \quad \theta \leq \theta_{\max}, \quad \text{MDA}(k) \text{ converged}$$

Result: $k^* \approx 55.353 \text{ N/m}$, achieved exactly at the constraint boundary $\theta = 10^\circ$. Physically: making the spring stiffer than necessary wastes material and weight; making it too soft allows the pendulum to deflect past the limit. The optimum is at the constraint boundary.

Spring-Pendulum Worked Example V

IDF Formulation for Spring-Pendulum (Exercise 24.5)

IDF adds coupling variables θ_c (theta sub c, the coupling copy of θ) and M_c (M sub c, the coupling copy of M) as optimizer decision variables alongside k :

$$\min_{k, \theta_c, M_c} k \quad \text{subject to} \quad k > 0, \quad \theta_c = F_{\text{disp}}(k, M_c), \quad M_c = F_{\text{loads}}(\theta_c), \quad \theta_c \leq \theta_{\text{max}}$$

The two coupling equality constraints enforce $\theta_c = M_c/k$ and $M_c = mg\ell \cos(\theta_c)$. The two disciplines (Loads and Displacement) can now be called in parallel since each receives a copy of the full assignment. The IDF solution gives the same $k^* \approx 55.353 \text{ N/m}$.

Python: Spring-Pendulum MDF Solution

```
import numpy as np
from scipy.optimize import minimize_scalar

m, ell, g = 1.0, 1.0, 9.81
theta_max = np.deg2rad(10.0)    # 10 degrees converted to radians

def loads_analysis(theta):
    """Compute bending moment  $M = m * g * ell * \cos(\theta)$ ."""
    return m * g * ell * np.cos(theta)

def displacement_analysis(M, k):
    """Compute deflection  $\theta = M / k$  (Hooke's law for torsional spring)."""
    return M / k

def run_mda_pendulum(k, tol=1e-10, max_iter=500):
    """
    Gauss-Seidel MDA for the spring-pendulum system.

    Given spring stiffness k, find the compatible (theta, M) pair by
    iterating:  $\theta \leftarrow M/k$ ,  $M \leftarrow m * g * ell * \cos(\theta)$ , until convergence.
    """
    theta = theta_max / 2.0    # initial guess: half the maximum deflection
    for iteration in range(max_iter):
        M = loads_analysis(theta)
        theta_new = displacement_analysis(M, k)
        if abs(theta_new - theta) < tol:
            return theta_new, M, True    # converged
```


CO Formulation for Spring-Pendulum (Exercise 24.6) I

The Collaborative Optimization formulation for the spring-pendulum problem provides the clearest illustration of how the CO architecture decomposes a coupled problem.

Global variables: $\mathcal{A}_g = \{k_g, \theta_g, M_g\}$ ($k_{\text{sub } g}$, $\theta_{\text{sub } g}$, $M_{\text{sub } g}$ — the system-level targets for spring stiffness, deflection angle, and moment respectively).

CO Formulation for Spring-Pendulum (Exercise 24.6) II

Displacement Sub-Problem

The “structures” sub-problem minimises how far the local displacement computation deviates from the system targets:

$$\min_{k, M} J_{\text{disp}}(k_g, \theta_g, M_g) = (k_g - k)^2 + (M_g - M)^2 + (F_{\text{disp}}(k_g, M_g) - \theta_g)^2$$

$$\text{subject to } \theta_g \leq \theta_{\max}, \quad k > 0$$

Here $(k_g - k)^2$ penalises deviation of the local stiffness from the global target; $(M_g - M)^2$ penalises deviation of the local moment from the global target; and $(F_{\text{disp}}(k_g, M_g) - \theta_g)^2$ penalises the residual of the displacement equation given the current targets. The sub-problem is solved by the structures team using their own constraints on k and M .

CO Formulation for Spring-Pendulum (Exercise 24.6) III

Loads Sub-Problem

The “loads” sub-problem minimises:

$$\min_{\theta_g} J_{\text{loads}}(\theta_g) = (\theta_g - \theta)^2 + (F_{\text{loads}}(\theta_g) - M)^2$$

where $F_{\text{loads}}(\theta_g) = mg\ell \cos(\theta_g)$ is the loads analysis. This sub-problem ensures that the loads team's local value of θ and its resulting moment M match the system targets.

CO Formulation for Spring-Pendulum (Exercise 24.6) IV

System-Level CO Problem

The system-level optimizer controls $\{k_g, \theta_g, M_g\}$ and minimises the spring stiffness (design goal) subject to both disciplines being able to match the targets:

$$\min_{k_g, \theta_g, M_g} k_g \quad \text{subject to} \quad J_{\text{disp}}^* = 0, \quad J_{\text{loads}}^* = 0$$

where J_{disp}^* and J_{loads}^* are the *optimal values* of the disciplinary sub-problems given the system targets. The constraints $J^* = 0$ enforce that the system targets are achievable by both disciplines simultaneously.

Key Insight from the CO Formulation

The CO formulation makes the separation of concerns explicit: the structures team owns J_{disp} and the loads team owns J_{loads} . Neither team needs to implement the other team's analysis. The system optimizer only sees the optimal coupling objective values J_i^* , which it uses as constraints. This is ideal for large organizations with separate engineering groups.

Comparing the MDO Architectures I

Having studied all five architectures in detail, we can now make a systematic comparison. The choice of architecture depends critically on the problem structure, computational resources, organizational constraints, and the availability of analytic derivatives.

Variable and Constraint Counts

Architecture	Variables	Eq. Constraints
MDF	n	0 (implicit in MDA)
Sequential	$n_g + \sum n_{\ell,i}$	0
IDF	$n + \sum n_i$	$\sum n_i$
CO	n_g (outer)	m ($J^* = 0$)
SAND	$n + \sum n_i$	$\sum n_i$

Here n is the total number of design variables, n_g is the number of global (shared) design variables, $n_{\ell,i}$ is the number of local variables for discipline i , and n_i is the number of response variables for discipline i .

Analysis Calls per Optimizer Iteration

Architecture	Calls per outer iteration
MDF	$K \cdot m$ (K = MDA iterations)
Sequential	1 per subproblem (sequential)
IDF	m (parallel)
CO	inner solve + m (parallel)
SAND	m (no inner loop)

MDF has the highest per-iteration cost because it runs the full MDA (K iterations) at every function evaluation. IDF and SAND require exactly m analysis calls per outer iteration (but SAND needs analytic Jacobians).

Comparing the MDO Architectures II

MDO Architecture Comparison

Architecture	Optimizer Level	MDA Required	Parallel Disciplines	Variable Count	Notes
MDF	Single	Yes (inner loop)	No	Low (n)	Simple; MDA cost dominates
Sequential	Hierarchical	Per discipline	No	Low-Med	Good for loosely coupled
IDF	Single	No	Yes	High ($n + \sum n_i$)	Coupling as constraint
CO	Bi-level	No	Yes	High	Each discipline independent
SAND	Single (large)	No (residuals)	Yes	Very high	Needs analytic gradients

When to use each architecture:

Use MDF when:

- The MDA converges in few iterations (loosely coupled system).
- You need physically feasible intermediates (e.g., for monitoring or early stopping).

Use SAND when:

- Analytic Jacobians (or an adjoint code) are available.
- The analysis code can be differentiated efficiently.
- Fastest wall-clock time is critical (e.g., real-time optimization).

Chapter 24 Summary I

This chapter has introduced the field of Multidisciplinary Design Optimization (MDO), the framework for optimising engineering systems in which multiple interacting analyses must be satisfied simultaneously. The following are the key ideas to carry away.

1. Assignments and Disciplinary Analyses

An assignment $\mathcal{A}$ collects all design variables $\mathbf{x}$ and response variables $\mathbf{y}^{(i)}$ into a single dictionary. A disciplinary analysis $\mathcal{F}_i$ reads the current assignment and overwrites the response variables of discipline i . In the three-discipline running example: $y^{(1)} = y^{(2)} - x$, $y^{(2)} = \sin(y^{(1)} + y^{(3)})$, $y^{(3)} = \cos(x + y^{(1)} + y^{(2)})$.

2. Interdisciplinary Compatibility and the MDA

An assignment is compatible if running any disciplinary analysis leaves it unchanged: $\mathcal{F}_i(\mathcal{A}) = \mathcal{A}$ for all i . The Gauss-Seidel MDA finds a compatible assignment iteratively. The ordering of analyses matters: a topologically sorted ordering converges faster and avoids oscillation.

3. The Five MDO Architectures

All five architectures solve the same underlying problem — minimise $f(\mathcal{A})$ subject to compatibility — but differ in how they handle the coupling:

- ① **MDF**: embed MDA inside the objective; optimizer sees a black box; every iterate is feasible.
- ② **Sequential**: round-robin discipline sub-problems; modular; no guarantee of system optimum.
- ③ **IDF**: coupling copies in optimizer; equality constraints enforce compatibility; disciplines run in parallel.
- ④ **CO**: bi-level; system optimizer sets targets; discipline sub-optimizers minimise coupling objectives.
- ⑤ **SAND**: all variables free; residuals are constraints; no inner loops; requires analytic Jacobians.

Chapter 24 Summary III

4. The Spring-Pendulum Worked Example

The spring-pendulum system — minimise spring stiffness k subject to $\theta \leq 10^\circ$ — concretely demonstrates all architectures. The MDA couples: $M = mg\ell \cos \theta$ (loads) and $\theta = M/k$ (displacement). All architectures converge to $k^* \approx 55.353$ N/m at $\theta^* = 10^\circ$ (the constraint boundary).

5. Architectural Trade-offs

Architecture	Feasible Iterates	Parallel Analyses	Variable Space	Analytic Jac. Needed?
MDF	Yes	No	Small	No
Sequential	Partial	Partial	Medium	No
IDF	No	Yes	Large	Helpful
CO	No	Yes	Large (inner)	No
SAND	No	Yes	Large	Yes

The Central Insight of MDO

There is no universally best architecture. The right choice depends on the coupling strength (how much disciplines affect one another), the cost of individual analyses, the availability of derivatives, organisational structure, and whether feasible intermediates are required. Understanding these trade-offs is the practical core of MDO.

Exercises I

Exercise 24.1: Real-World MDO Example

Provide an example of a real-world multidisciplinary design optimization problem involving at least three disciplines. Draw the dependency diagram. Identify which pairs of disciplines are *tightly coupled* (strong circular dependency) and which are *loosely coupled* (one-way or weak dependency). Based on your dependency diagram, which MDO architecture would you recommend, and why?

Exercise 24.2: Effect of Analysis Ordering

Consider the three-discipline system from Example 24.2:

$$y^{(1)} = y^{(2)} - x, \quad y^{(2)} = \sin(y^{(1)} + y^{(3)}), \quad y^{(3)} = \cos(x + y^{(1)} + y^{(2)})$$

Implement the Gauss-Seidel MDA in Python and run it for $x = 1$ with (a) ordering F_1, F_2, F_3 and (b) ordering F_1, F_3, F_2 . Plot the values of $y^{(1)}, y^{(2)}, y^{(3)}$ vs. iteration number for both orderings. Explain, using the dependency graph, why one ordering converges and the other does not.

Exercise 24.3: Topological Sorting

Given the ride-sharing dependency diagram (Vehicle $\rightarrow$ Sensor, Sensor $\rightarrow$ Vehicle (cycle), Sensor $\rightarrow$ Autonomy, Vehicle $\rightarrow$ Autonomy, Autonomy $\rightarrow$ Routing, Routing $\rightarrow$ Demand (cycle), Demand $\rightarrow$ Routing, Demand $\rightarrow$ Profit, Routing $\rightarrow$ Profit): (a) Identify all strongly connected components (groups of disciplines that mutually depend on one another). (b) For each pair of tightly coupled analyses, what initial guess strategy would you use to start the MDA?

Exercises III

Exercise 24.4: Spring-Pendulum MDF

Formulate the spring-pendulum problem under the MDF architecture for $m = 1 \text{ kg}$, $\ell = 1 \text{ m}$, $g = 9.81 \text{ m/s}^2$, $\theta_{\max} = 10^\circ$. Implement the MDA and the outer optimizer in Python using `scipy.optimize.minimize_scalar`. Verify that $k^* \approx 55.353 \text{ N/m}$ and that the constraint $\theta \leq \theta_{\max}$ is active at the optimum. What does “active constraint” mean physically for the spring design?

Exercise 24.5: Spring-Pendulum IDF

Reformulate the spring-pendulum problem under the IDF architecture. Identify the coupling variables (the optimizer-controlled copies of the response variables) and write down the two coupling equality constraints explicitly. Implement the IDF formulation in Python using `scipy.optimize.minimize` with `method='SLSQP'` and `NonlinearConstraint`. Verify that the IDF solution agrees with the MDF solution ($k^* \approx 55.353 \text{ N/m}$) and that the coupling residuals are essentially zero at the solution.

Exercise 24.6: Spring-Pendulum CO

Formulate the spring-pendulum problem under the Collaborative Optimization architecture. Write out explicitly: (a) the displacement sub-problem (minimising J_{disp}), (b) the loads sub-problem (minimising J_{loads}), and (c) the system-level problem (minimise k_g subject to $J_{\text{disp}}^* = 0$ and $J_{\text{loads}}^* = 0$). What are the global variables $\mathcal{A}_g$? Discuss: what advantage does the CO decomposition offer if the loads team uses a proprietary finite-element solver and the structures team uses a proprietary stress-analysis code?

Further Reading and References

-  M. J. Kochenderfer and T. A. Wheeler, *Algorithms for Optimization*, 2nd ed., MIT Press, 2026. (Chapter 24.)
-  J. R. R. A. Martins and A. B. Lambe, “Multidisciplinary design optimization: A survey of architectures,” *AIAA Journal*, 51(9):2049–2075, 2013. The authoritative survey paper on MDO architectures; highly recommended for practitioners.
-  E. J. Cramer, J. E. Dennis, P. D. Frank, R. M. Lewis, and G. R. Shubin, “Problem formulation for multidisciplinary optimization,” *SIAM Journal on Optimization*, 4(4):754–776, 1994. The original paper that formalised MDF, IDF, and SAND architectures.
-  R. D. Braun and I. M. Kroo, “Use of the collaborative optimization architecture for launch vehicle design,” *AIAA/NASA 6th Symposium on Multidisciplinary Analysis and Optimization*, 1996. The paper that introduced and validated the CO architecture on a real aerospace design problem.
-  A. B. Lambe and J. R. R. A. Martins, “Extensions to the design structure matrix for the description of multidisciplinary design, analysis, and optimization processes,” *Structural and Multidisciplinary Optimization*, 46(2):273–284, 2012. Introduces the XD SM (Extended Design Structure Matrix) notation widely used to document MDO architectures.